UNIT-5 MUTABLE DATA STRUCTURES

LIST

List is a data type where you can store multiple items under 1 name. More technically, lists act like dynamic arrays which

means you can add more items on the fly.

Lists are heterogenous data structures created by elements separated with commas and enclosed within square brackets.

Array Vs Lists

Convenience -> Hetrogeneous

Speed of Execution(list slow)

Memory(list require more space)

Characterstics of a List

Ordered Changeble/Mutable Hetrogeneous

Can have duplicates are dynamic

can be nested

items can be accessed

can contain any kind of objects in python

In [1]:
```
#ordered
l=[1,2,3]
l1=[3,2,1]
l==l1
```

Out[1]:
```
False
```

In [2]:
```
#changable/mutable
l=[1,2,3]
l[0]=5
print(l)
```

```
[5, 2, 3]
```

In [3]:
```
#Hetrogeneous
l=[1,1.0,"apple",(9,0),{1,0},{"a":1,"b":3}]
print(l)
```

```
[1, 1.0, 'apple', (9, 0), {0, 1}, {'a': 1, 'b': 3}]
```

In [4]:
```
#duplicate
l=[1,1,1,2,2,3,3,4,4,5,5,6]
print(l)
```

```
[1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6]
```

In [5]:
```
#dynamic
l=[1,2,3]
l.append(5)
l.append((1,2))
print(l)
```

```
[1, 2, 3, 5, (1, 2)]
```

In [6]:
```
#can be nested
l=[1,2,3,[1,2]]
print(l)
```

```
[1, 2, 3, [1, 2]]
```

In [7]:
```
#items can be accessed
l=[1,2,3,[1,2]]
print(l[0])
print(l[3][1])
```

```
1
2
```

In [9]:
```
#empty list
l=[]
print(l)
l=list()
print(l)
l1=list((1,2,3,4))
print(l1)
```

```
[]
[]
[1, 2, 3, 4]
```

In [10]:
```
#1d list
l=[1,2,3]
l1=["a","b","c"]
l2=[(1,2),(3,4),(5,6)]
l3=[1,'a',(1,2),{1,2}]
print(l,l1,l2,l3)
#2D list
l=[[1,2],[3,4]]
print(l)
#3d list
l=[[[1,2],[3,4]],[[5,6],[7,8]]]
print(l)
```

```
[1, 2, 3] ['a', 'b', 'c'] [(1, 2), (3, 4), (5, 6)] [1, 'a', (1, 2), {1, 2}]
[[1, 2], [3, 4]]
[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
```

In [11]:
```
#using type conversation
l=list("hello")
print(l)
```

```
['h', 'e', 'l', 'l', 'o']
```

In [12]:
```
l1=list((1,2,3,4))
print(l1)
```

```
[1, 2, 3, 4]
```

In [13]:
```
l1=list({1,2,3})
print(l1)
```

```
[1, 2, 3]
```

Accessing items from a list

indexing

slicing

In [14]:
```
#posative indexing
l=[1,2,3,4]
print(l[0])
print(l[2])
print(l[4])
```

```
1
3
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-14-9a5c61c28a39> in <module>
      3 print(l[0])
      4 print(l[2])
----> 5 print(l[4])

IndexError: list index out of range
```

In [15]:
```
#negative indexing
l=[1,2,3,4]
print(l[-1])
print(l[-2])
print(l[-4])
print(l[-5])
```

```
4
3
1

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-15-162553cda0c1> in <module>
      4 print(l[-2])
      5 print(l[-4])
----> 6 print(l[-5])

IndexError: list index out of range
```

In [16]:
```
l=[1,2,3,[1,2],"abcd"]
print(l[3][-1])
print(l[3][1])
print(l[4][:2])
```

```
2
2
ab
```

In [17]:
```
#slicing
l=[1,2,3,4]
print(l[::2])
print(l[1:3])
print(l[-1:-4:-2])
```

```
[1, 3]
[2, 3]
[4, 2]
```

In [18]:
```
l=[1,2,3,[1,2],"abcd"]
print(l[1:4])
print(l[2:5])
print(l[2:5][2][1:2])
```

```
[2, 3, [1, 2]]
[3, [1, 2], 'abcd']
b
```

In [23]:
```
l=[1,2,3,[1,2],"abcd"]
print(l[-1:-3:-1][-1][-2])
l=[1,2,3,4]
print(l[::-1])
```

```
1
[4, 3, 2, 1]
```

Adding Items to a List

append, extend ,insert

In [24]:
```
# append
L = [1,2,3,4,5]
L.append(True)
print(L)
```

```
[1, 2, 3, 4, 5, True]
```

In [28]:
```
# append
L = [1,2,3,4,5]
L.append([1,2,3])
print(L)
# append
L = [1,2,3,4,5]
print(L.append("abcdf"))
print(L)
# append
L = [1,2,3,4,5]
L.append(1,2)
print(L)
```

```
[1, 2, 3, 4, 5, [1, 2, 3]]
None
[1, 2, 3, 4, 5, 'abcdf']

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-28-8ccd40d4e0a2> in <module>
      9 # append
     10 L = [1,2,3,4,5]
---> 11 L.append(1,2)
     12 print(L)

TypeError: append() takes exactly one argument (2 given)
```

In [29]:
```
# extend
L = [1,2,3,4,5]
L.extend([6,7,8])
print(L)
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

In [30]:
```
# extend
L = [1,2,3,4,5]
L.extend("hello")
print(L)
```

```
[1, 2, 3, 4, 5, 'h', 'e', 'l', 'l', 'o']
```

In [31]:
```
# extend
L = [1,2,3,4,5]
L.extend(range(10))
print(L)
```

```
[1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [32]:
```
# extend
L = [1,2,3,4,5]
print(L.extend([6,7,[1,8,"a"]]))
print(L)
```

```
None
[1, 2, 3, 4, 5, 6, 7, [1, 8, 'a']]
```

In [33]:
```
# extend
L = [1,2,3,4,5]
L.extend(1,2,3)
print(L)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-33-2834b8e71b0c> in <module>
      1 # extend
      2 L = [1,2,3,4,5]
----> 3 L.extend(1,2,3)
      4 print(L)

TypeError: extend() takes exactly one argument (3 given)
```

In [34]:
```
L = [1,2,3,4,5]
L.insert(1,100)
print(L)
```

```
[1, 100, 2, 3, 4, 5]
```

In [36]:
```
# insert
L = [1,2,3,4,5]
L.insert(1,{5:4,7:5})
print(L)
# insert
L = [1,2,3,4,5]
print(L.insert(2,"helo"))
print(L)
```

```
[1, {5: 4, 7: 5}, 2, 3, 4, 5]
None
[1, 2, 'helo', 3, 4, 5]
```

Editing items in a List

editing with indexing , editing with slicing

In [37]:
```
L = [1,2,3,4,5]
# editing with indexing
L[-1] = 500
# editing with slicing
L[1:4] = [200,300,400]
print(L)
```

```
[1, 200, 300, 400, 500]
```

Deleting items from a List

Deleting list elements When deleting a list element, you can use either of the del statements

to know exactly which element(s) you are deleting.

You can use the remove() method if you don't know exactly which items to delete.

Rundown can be replaced with the collection as the development community

is familiar with this term rather than the name rundown.

del , remove, pop , clear

In [38]:
```
L = [1,2,3,4,5]
# indexing
del L[-1]
# slicing
del L[1:3]
print(L)
```

```
[1, 4]
```

In [39]:
```
# remove
L = [1,2,3,4,5]
L.remove(5)
print(L)
```

```
[1, 2, 3, 4]
```

In [40]:
```
L = [1,2,3,4,5]
# slicing
del L[1:7]
print(L)
```

```
[1]
```

In [44]:
```
# remove
L = [1,2,3,4,5,2]
L.remove(2)
print(L)
L.remove(6)
print(L)
```

```
[1, 3, 4, 5, 2]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-44-e8590105f5ec> in <module>
      4 print(L)
      5
----> 6 L.remove(6)
      7 print(L)

ValueError: list.remove(x): x not in list
```

In [46]:
```
# pop
L = [1,2,3,4,5]
L.pop()
print(L)
L.pop(2) #2c is index no.
print(L)
# pop
L.pop(6)
print(L)
```

```
[1, 2, 3, 4]
[1, 2, 4]

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-46-57d1a3bbfbbe> in <module>
      6 print(L)
      7 # pop
----> 8 L.pop(6)
      9 print(L)

IndexError: pop index out of range
```

In [47]:
```
# clear
L = [1,2,3,4,5]
L.clear()
print(L)
```

```
[]
```

Operations on Lists

Arithmetic, Membership , Loop

In [48]:
```
# Arithmetic (+ ,*)

L1 = [1,2,3,4]
L2 = [5,6,7,8]
# Concatenation/Merge
print(L1 + L2)
print(L1*3)
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4]
```

In [49]:
```
#Membership
L1 = [1,2,3,4,5]
L2 = [1,2,3,4,[5,6]]
print(5 not in L1)
print([5,6] in L2)
print(5 not in L2)
```

```
False
True
True
```

In [51]:
```
l = ["apple", "banana", "cherry"]
i = 0
while i < len(l):
    print(l[i])
    i = i + 1
```

```
apple
banana
cherry
```

In [52]:
```
L1 = [1,2,3,4,5]
for i in L1:
    print(i)
```

```
1
2
3
4
5
```

In [53]:
```
L3 = [[[1,2],[3,4]],[[5,6],[7,8]]]
for i in L3:
    print(i)
```

```
[[1, 2], [3, 4]]
[[5, 6], [7, 8]]
```

In [54]:
```
L1 = [1,2,3,4,5]
for i,j in enumerate(L1):
    print(i,j)
```

```
0 1
1 2
2 3
3 4
4 5
```

List function

len ,min ,max, sum, sort, sorted, count, index, reverse, sort vs sorted ,copy

In [56]:
```
L = [2,1,5,7,0]
print(len(L))
L = [2,1,5,7,0,(7,8),[1,2,3],{1,2,3},{1:2,3:4}]
print(len(L))
```

```
5
9
```

In [57]:
```
l=[1,2,3,4,5]
print(max(l))
```

```
5
```

In [58]:
```
l=[[1,2],[3,4],[5,6]]
print(max(l))
```

```
[5, 6]
```

In [59]:
```
l=[1,2,3,4,5,[1,2]]
print(max(l))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-59-a6a0359c1acb> in <module>
      1 l=[1,2,3,4,5,[1,2]]
----> 2 print(max(l))

TypeError: '>' not supported between instances of 'list' and 'int'
```

In [60]:
```
l=[[1,2],[3,4],[5,4]]
print(max(l))
```

```
[5, 4]
```

In [61]:
```
l=[1,2,3,4,5]
print(min(l))
```

```
1
```

In [62]:
```
l=["abv","k","k"]
print(min(l))
```

```
abv
```

In [63]:
```
print(sum[1,2,3])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-63-37595b457d77> in <module>
----> 1 print(sum[1,2,3])

TypeError: 'builtin_function_or_method' object is not subscriptable
```

In [66]:
```
l=[1,2,3]
print(sum(l))
l=['a','b']
print(l[0]+l[1])
print(sum(l))
```

```
6
ab

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-66-53cb4dd1375d> in <module>
      3 l=['a','b']
      4 print(l[0]+l[1])
----> 5 print(sum(l))

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [67]:
```
l=[1,5,3]
print(l.sort())
print(l)
print(sorted(l,reverse=True))
```

```
None
[1, 3, 5]
[5, 3, 1]
```

In [70]:
```
l=[[1,2],[5,4],[3,6]]
l.sort()
print(l)
print(sorted(l,reverse=False))
l=[1,"m",2]
print(sorted(l,reverse=False))
```

```
[[1, 2], [3, 6], [5, 4]]
[[1, 2], [3, 6], [5, 4]]

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-70-2d43747a997f> in <module>
      4 print(sorted(l,reverse=False))
      5 l=[1,"m",2]
----> 6 print(sorted(l,reverse=False))

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [71]:
```
# sort (vs sorted)
L = [2,1,5,7,0]
print(L)
print(sorted(L))
print(L)
L.sort()
print(L)
```

```
[2, 1, 5, 7, 0]
[0, 1, 2, 5, 7]
[2, 1, 5, 7, 0]
[0, 1, 2, 5, 7]
```

In [73]:
```
l=[1,2,1,"a",(1,2),(3,4),(1,2)]
print(l.count(1))
print(l.count('a'))
print(l.count((1,2)))
```

```
2
1
2
```

In [74]:
```
# index
L = [1,2,1,3,4,1,5]
L.index(1)
```

Out[74]:
```
0
```

In [75]:
```
L = [1,2,1,3,4,1,5]
L.index(1,1,9)
```

Out[75]:
```
2
```

In [76]:
```
# index
L = [1,2,1,3,4,1,(1,5)]
L.index((1,5))
```

Out[76]:
```
6
```

In [77]:
```
L = [1,2,1,3,4,1,5,[6,4]]
L.index(6)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-77-10bfb5e6aa85> in <module>
      1 L = [1,2,1,3,4,1,5,[6,4]]
----> 2 L.index(6)

ValueError: 6 is not in list
```

In [79]:
```
# reverse
L = [2,1,5,7,0]
# permanently reverses the list
print(L.reverse() )
print(L)
```

```
None
[0, 7, 5, 1, 2]
```

In [81]:
```
# copy -> shallow
L = [2,1,5,7,0]
print(L)
print(id(L))
L1 = L.copy()
print(L1)
print(id(L1))
```

```
[2, 1, 5, 7, 0]
2125476033536
[2, 1, 5, 7, 0]
2125476201664
```

In [82]:
```
# copy -> shallow
L = [2,1,5,7,0]
print(L)
print(id(L))
L1 = L[::]
print(L1)
print(id(L1))
```

```
[2, 1, 5, 7, 0]
2125475267968
[2, 1, 5, 7, 0]
2125474702336
```

In [84]:
```
L = [2,1,5,7,0]
print(L)
print(id(L))
L1 = L
print(L1)
print(id(L1))
```

```
[2, 1, 5, 7, 0]
2125474700544
[2, 1, 5, 7, 0]
2125474700544
```

In [86]:
```
a=5
print(a)
print(id(a))
b=a
print(b)
print(id(b))
a+=5
print(a)
print(b)
```

```
5
140726356813728
5
140726356813728
10
5
```

List Comprehension

List Comprehension provides a concise way of creating lists.

newlist = [expression for item in iterable if condition == True]

Advantages of List Comprehension

More time-efficient and space-efficient than loops. Require fewer lines of code.

Transforms iterative statement into a formula.

In [87]:
```
# Add 1 to 10 numbers to a list
L = []
for i in range(1,11):
    L.append(i)
print(L)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [88]:
```
L = [i for i in range(1,11)]
print(L)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [89]:
```
# scalar multiplication on a vector
v = [2,3,4]
s = -3
# [-6,-9,-12]
[s*i for i in v]
```

Out[89]:
```
[-6, -9, -12]
```

In [1]:
```
# Print all numbers divisible by 5 in the range of 1 to 50

[i for i in range(1,51) if i%5 == 0]
```

Out[1]:
```
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
```

In [2]:
```
# find languages which start with letter p
languages = ['java','python','php','c','javascript']

[language for language in languages if language.startswith('p')]
```

Out[2]:
```
['python', 'php']
```

In [4]:
```
# Nested if with List Comprehension
basket = ['apple','guava','cherry','banana']
my_fruits = ['apple','kiwi','grapes','banana']

# add new list from my_fruits and items if the fruit exists in basket and al

[fruit for fruit in my_fruits if fruit in basket if fruit.startswith('a')]
```

Out[4]:
```
['apple']
```

In [5]:
```
# Print a (3,3) matrix using list comprehension -> Nested List comprehension
[[i*j for i in range(1,4)] for j in range(1,4)]
```

Out[5]:
```
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

In [6]:
```
# cartesian products -> List comprehension on 2 lists together
L1 = [1,2,3,4]
L2 = [5,6,7,8]

[i*j for i in L1 for j in L2]
```

Out[6]:
```
[5, 6, 7, 8, 10, 12, 14, 16, 15, 18, 21, 24, 20, 24, 28, 32]
```

3 ways to traverse a list

itemwise, indexwise, indexwise with index

In [7]:
```
# itemwise
L = [1,2,3,4]
for i in L:
    print(i)
```

```
1
2
3
4
```

In [8]:
```
# indexwise
L = [1,2,3,4]
for i in range(0,len(L)):
    print(L[i])
```

```
1
2
3
4
```

In [10]:
```
# itemwise with indexwise
L = [1,2,3,4]
for i,j in enumerate(L):
    print(i,"--",j)
```

```
0 -- 1
1 -- 2
2 -- 3
3 -- 4
```

Zip

The zip() function returns a zip object, which is an iterator of tuples where the first item in each passed iterator is paired together, and then the second item in each passed iterator are paired together.

If the passed iterators have different lengths, the iterator with the least items decides the length of the new iterator.

In [11]:
```
# Write a program to add items of 2 lists indexwise

L1 = [1,2,3,4]
L2 = [-1,-2,-3,-4]

list(zip(L1,L2))

[i+j for i,j in zip(L1,L2)]
```

Out[11]:
```
[0, 0, 0, 0]
```

In [12]:
```
a = [1,2,3]
b = a.copy()

print(a)
print(b)

a.append(4)
print(a)
print(b)

# lists are mutable
```

```
[1, 2, 3]
[1, 2, 3]
[1, 2, 3, 4]
[1, 2, 3]
```

In [13]:
```
# Write code here
list1 = ["M", "na", "i", "Kh"]
list2 = ["y", "me", "s", "an"]

[[i,j] for (i,j) in zip(list1,list2)]
```

Out[13]:
```
[['M', 'y'], ['na', 'me'], ['i', 's'], ['Kh', 'an']]
```

Problem

Running Sum on list Write a program to print a list after performing running sum on it. i.e:

Input:

list1 = [1,2,3,4,5,6] Output:

[1,3,6,10,15,21]

In [17]:
```
list1 = [1,2,3,4,5,6]
result = []
sum = 0
for i in list1:
    sum = sum + i
    result.append(sum)
print(result)
```

```
[1, 3, 6, 10, 15, 21]
```

Problem

You are given a list of integers. You are asked to make a list by running through elements of the list by adding all elements greater and itself.

i.e. Say given list is [2,4,6,10,1] resultant list will be [22,20,16,10,23] .

For 1st element 2 ->> these are greater (4+6+10) values and 2 itself so on adding becomes 22 .

For 2nd element 4 ->> greater elements are (6, 10) and 4 itself, so on adding 20

like wise for all other elememts.

[2,4,6,10,1] --> [22,20,16,10,23]

In [22]:
```
# Write code here
L = [2,4,6,10,1]
result = []

for i in L:
    sum = 0
    for j in L:
        if i <= j:
            sum = sum + j
    result.append(sum)
print(result)
```

```
[22, 20, 16, 10, 23]
```

Problem

Find list of common unique items from two list. and show in increasing order

Input

num1 = [23,45,67,78,89,34] num2 = [34,89,55,56,39,67]

Output:

[34, 67, 89]

In [24]:
```
# Write code here
num1 = [23,45,67,78,89,34,67]
num2 = [34,89,55,56,39,67,67]

common = []

for i in num1:
    if i in num2:
        if i not in common:
            common.append(i)
print(common)
```

```
[67, 89, 34]
```

Problem

Sort a list of alphanumeric strings based on product value of numeric character in it. If in any string there is no numeric character take it's product value as 1. Input:

['1ac21', '23fg', '456', '098d','1','kls']

Output:

['456', '23fg', '1ac21', '1', 'kls', '098d']

In [27]:
```
# Write code here
L = ['1ac21', '23fg', '456', '098d','1','kls']
prod_value = []
for item in L:
    product = 1
    for char in item:
        if char.isdigit():
            product = product*int(char)
    prod_value.append(product)
print(prod_value)
print([i for i in sorted(list(zip(prod_value,L)),reverse=True)])
[i[1] for i in sorted(list(zip(prod_value,L)),reverse=True)]
```

```
[2, 6, 120, 0, 1, 1]
[(120, '456'), (6, '23fg'), (2, '1ac21'), (1, 'kls'), (1, '1'), (0, '098
d')]
```

Out[27]:
```
['456', '23fg', '1ac21', 'kls', '1', '098d']
```

Problem

Split String of list on K character. Example :

Input:

['CampusX is a channel', 'for data-science', 'aspirants.']

Output:

['CampusX', 'is', 'a', 'channel', 'for', 'data-science', 'aspirants.']

In [30]:
```
# Write code here
L = ['CampusX is a channel', 'for data-science', 'aspirants.']
inp = 'a'
result = []
for i in L:
    result.extend(i.split(inp))
print(result)
```

```
['C', 'mpusX is ', ' ch', 'nnel', 'for d', 't', '-science', '', 'spir', 'nt
s.']
```

In [35]:
```
#Write a list comprehension to print the following matrix [[0, 1, 2], [3, 4,
[[j + 3*i for j in range(1,4)] for i in range(0,3)]
```

Out[35]:
```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

In [36]:
```
#Write a list comprehension that can transpose a given matrix matrix = [ [1,
# Write code here
matrix = [ [1,2,3],
[4,5,6],
[7,8,9]
]

[[row[i] for row in matrix]for i in range(len(matrix))]
```

Out[36]:
```
[[1, 4, 7], [2, 5, 8], [3, 6, 9]]
```

In [37]:
```
matrix = [ [1,2,3],
[4,5,6],
[7,8,9]
]

[item for row in matrix for item in row]
```

Out[37]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [39]:
```
l=[]
for i in matrix:
    for j in i:
        l.append(j)
print(l)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dictionaries

Dictionary in Python is a collection of keys values, used to store data values like a map, which,

unlike other data types which hold only a single value as an element.

In some languages it is known as map or assosiative arrays.

dict = { 'name' : 'nitish' , 'age' : 33 , 'gender' : 'male' }

Characterstics:Mutable

Indexing has no meaning keys can't be duplicated keys can't be mutable items

Creating a dictionary

Curly brackets represent dictionary entries. Key-value pairs are separated by a colon in the dictionary.

An empty dictionary without any items is written with just two curly braces, i.e.,

In [41]:
```
# empty dictionary
d = {}
d
```

Out[41]:
```
{}
```

In [42]:
```
# 1D dictionary
d1 = { 'name' : 'nitish' ,'gender' : 'male' }
d1
```

Out[42]:
```
{'name': 'nitish', 'gender': 'male'}
```

In [44]:
```
# with mixed keys
d2 = {(1,2,3):1,'hello':'world'}
d2
```

Out[44]:
```
{(1, 2, 3): 1, 'hello': 'world'}
```

In [45]:
```
d2 = {[1,2,3]:1,'hello':'world'}
d2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-45-761024724dff> in <module>
----> 1 d2 = {[1,2,3]:1,'hello':'world'}
      2 d2

TypeError: unhashable type: 'list'
```

In [46]:
```
# 2D dictionary -> JSON
s = {
'name':'nitish',
'college':'bit', 'sem':4,
'subjects':{
'dsa':50,
'maths':67, 'english':34
}
}
s
```

Out[46]:
```
{'name': 'nitish',
 'college': 'bit',
 'sem': 4,
 'subjects': {'dsa': 50, 'maths': 67, 'english': 34}}
```

In [47]:
```
# using sequence and dict function
d4 = dict([('name','nitish'),('age',32),(3,3)])
d4
```

Out[47]:
```
{'name': 'nitish', 'age': 32, 3: 3}
```

In [50]:
```
d5 = {'name':'nitish','name':'rahul'}
d5
```

Out[50]:
```
{'name': 'nitish', 'name1': 'rahul'}
```

In [51]:
```
d5 = {'name':'nitish','name1':'rahul'}
d5
```

Out[51]:
```
{'name': 'nitish', 'name1': 'rahul'}
```

Accessing the elements in a dictionary

You use the square brackets along with the key to access the elements of a dictionary.

The get() method returns a value for the given key; it returns default value None if the key is not available.

In [53]:
```
my_dict = {'name': 'Jack', 'age': 26}
my_dict['age']
```

Out[53]:
```
26
```

In [56]:
```
my_dict = {'name': 'Jack', 'age': 26}
print(my_dict['age'])
print(my_dict.get('age'))
```

```
26
26
```

In [57]:
```
s = {
'name':'nitish',
'college':'bit', 'sem':4,
'subjects':{
'dsa':50,
'maths':67, 'english':34
}
}
s['subjects']['maths']
```

Out[57]:
```
67
```

Updating a dictionary

Dictionary entries can be changed. We can add new items or change the values

of the existing ones using the assignment operator. The value is updated if the key already exists;

otherwise, a new key-value pair is added to the dictionary.

In [60]:
```
d4 = dict([('name','nitish'),('age',32),(3,3)])
d4['gender'] = 'male'
d4
```

Out[60]:
```
{'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male'}
```

In [61]:
```
d4['weight'] = 72
d4
```

Out[61]:
```
{'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male', 'weight': 72}
```

Deleting dictionary elements

The pop() method is used to remove an item from a dictionary.

It returns the value of an item removed with the provided key.

You can use popitem() to remove and return an arbitrary item (key, value) from a dictionary.

The clear() method is used to remove all items at once.

The del keyword can also be used to remove individual items or the entire dictionary.

pop, popitem, del, clear

In [66]:
```
d = {'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male', 'weight': 72}
# pop
d.pop(3)
print(d)
```

```
{'name': 'nitish', 'age': 32, 'gender': 'male', 'weight': 72}
```

In [67]:
```
d.popitem()
d.popitem()
print(d)
```

```
{'name': 'nitish', 'age': 32}
```

In [69]:
```
# del
d = {'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male', 'weight': 72}
del d['name']
print(d)
d.clear()
print(d)
```

```
{'age': 32, 3: 3, 'gender': 'male', 'weight': 72}
{}
```

Properties of dictionary keys

There are no restrictions on dictionary values.

A Python object can either be a standard object or a user-defined object,

but this is not true for dictionary keys. The two important properties of keys are as follows:

There will be no more than one entry per key, so duplicate keys are not allowed.

If duplicate keys are encountered during assignment, the last assignment wins. A key must be immutable type,

such as a dictionary key. You can use strings, numbers, or tuples, but a list, such as ['key'], is an invalid key.

In [70]:
```
s = {
'name':'nitish',
'college':'bit', 'sem':4,
'subjects':{
'dsa':50,
'maths':67, 'english':34
}
}
"name" in s
```

Out[70]:
```
True
```

In [71]:
```
s = {
'name':'nitish',
'college':'bit', 'sem':4,
'subjects':{
'dsa':50,
'maths':67, 'english':34
}
}
"nitish" in s
```

Out[71]:
```
False
```

In [72]:
```
d = {'name':'nitish','gender':'male','age':33}
for i in d:
    print(i,d[i])
```

```
name nitish
gender male
age 33
```

Dictionary function

len, sorted ,max ,min ,items, keys,values, update ,formkeys ,ksetdefualt

In [79]:
```
d = {'name':'nitish','gender':'male','age':33}
d1={"b":5,'a':12}
print(len(d))
print(sorted(d))
print(sorted(d1))
print(sorted(d,reverse=True))
print(max(d))
print(min(d))
print(min(d1))
```

```
3
['age', 'gender', 'name']
['a', 'b']
['name', 'gender', 'age']
name
age
a
```

In [81]:
```
d = {'name':'nitish','gender':'male','age':33}
print(d.keys())
for i in d.keys():
    print(i)
```

```
dict_keys(['name', 'gender', 'age'])
name
gender
age
```

In [82]:
```
d = {'name':'nitish','gender':'male','age':33}
print(d.values())
for i in d.values():
    print(i)
```

```
dict_values(['nitish', 'male', 33])
nitish
male
33
```

In [84]:
```
d = {'name':'nitish','gender':'male','age':33}
print(d.items())
for i,j in d.items():
    print(i,"-",j)
```

```
dict_items([('name', 'nitish'), ('gender', 'male'), ('age', 33)])
name - nitish
gender - male
age - 33
```

In [87]:
```
# update
d1 = {1:2,3:4,8:5}
d2 = {4:7,6:8}
d1.update(d2)
print(d1)
```

```
{1: 2, 3: 4, 8: 5, 4: 7, 6: 8}
```

setdefault()

The setdefault() method is similar to but it will set if the key is not already in dict. Syntax: dict.setdefault(key, default=None) Parameters

key – The key to search

Default - The value to be returned if the key cannot be found

Return value The method returns the value of the given key if it is present in the dictionary; it returns the default value if it isn't present

In [88]:
```
d = {'name':'nitish','gender':'male','age':33}
d.setdefault("age",55)
print(d)
```

```
{'name': 'nitish', 'gender': 'male', 'age': 33}
```

In [89]:
```
d = {'name':'nitish','gender':'male','age':33}
d.setdefault("age1",55)
print(d)
```

```
{'name': 'nitish', 'gender': 'male', 'age': 33, 'age1': 55}
```

In [16]:
```
seq = {'a', 'b', 'c', 'd', 'e'}
# creating dict with default values as None
res_dict = dict.fromkeys(seq)
print("The newly created dict with None values : " + str(res_dict))
# creating dict with default values as 1
res_dict2 = dict.fromkeys(seq, 1)

print("The newly created dict with 1 as value : " + str(res_dict2))
```

```
The newly created dict with None values : {'c': None, 'b': None, 'e': None,
'd': None, 'a': None}
The newly created dict with 1 as value : {'c': 1, 'b': 1, 'e': 1, 'd': 1,
'a': 1}
```

Dictionary Comprehension

In [90]:
```
# print 1st 10 numbers and their squares
{i:i**2 for i in range(1,11)}
```

Out[90]:
```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
```

In [91]:
```
# using zip
days = ["Sunday", "Monday","Tuesday","Wednesday","Thursday","Friday","Saturd
temp_C = [30.5,32.6,31.8,33.4,29.8,30.2,29.9]
{i:j for (i,j) in zip(days,temp_C)}
```

Out[91]:
```
{'Sunday': 30.5,
 'Monday': 32.6,
 'Tuesday': 31.8,
 'Wednesday': 33.4,
 'Thursday': 29.8,
 'Friday': 30.2,
 'Saturday': 29.9}
```

In [93]:
```
# Nested Comprehension
# print tables of number from 2 to 10
{i:{j:i*j for j in range(1,11)} for i in range(2,11)}
```

Out[93]:
```
{2: {1: 2, 2: 4, 3: 6, 4: 8, 5: 10, 6: 12, 7: 14, 8: 16, 9: 18, 10: 20},
 3: {1: 3, 2: 6, 3: 9, 4: 12, 5: 15, 6: 18, 7: 21, 8: 24, 9: 27, 10: 30},
 4: {1: 4, 2: 8, 3: 12, 4: 16, 5: 20, 6: 24, 7: 28, 8: 32, 9: 36, 10: 40},
 5: {1: 5, 2: 10, 3: 15, 4: 20, 5: 25, 6: 30, 7: 35, 8: 40, 9: 45, 10: 50},
 6: {1: 6, 2: 12, 3: 18, 4: 24, 5: 30, 6: 36, 7: 42, 8: 48, 9: 54, 10: 60},
 7: {1: 7, 2: 14, 3: 21, 4: 28, 5: 35, 6: 42, 7: 49, 8: 56, 9: 63, 10: 70},
 8: {1: 8, 2: 16, 3: 24, 4: 32, 5: 40, 6: 48, 7: 56, 8: 64, 9: 72, 10: 80},
 9: {1: 9, 2: 18, 3: 27, 4: 36, 5: 45, 6: 54, 7: 63, 8: 72, 9: 81, 10: 90},
 10: {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60, 7: 70, 8: 80, 9: 90, 10: 10
0}}
```

program

Replace words from Dictionary. Given String, replace it's words from lookup dictionary. Example 1:

Input:

test_str = 'CampusX best for DS students.'

repl_dict = {"best" : "is the best channel", "DS" : "Data-Science"} Output: CampusX is the best channel for Data-Science students. Example 2: Input:

test_str = 'CampusX best for DS students.'

repl_dict = {"good" : "is the best channel", "ds" : "Data-Science"} Output: CampusX best for DS students.

In [94]:
```
test_str = 'CampusX best for DS students.'
repl_dict = {"best" : "is the best channel", "DS" : "Data-Science"}

res = []
for i in test_str.split():
    if i in repl_dict:
        res.append(repl_dict[i])
    else:
        res.append(i)
print(" ".join(res))
```

```
CampusX is the best channel for Data-Science students.
```

program

Convert List to List of dictionaries. Given list values and keys list,

convert these values to key value pairs in form of list of dictionaries. Example 1:

Input:

test_list = ["DataScience", 3, "is", 8] key_list = ["name", "id"] Output:

[{'name': 'DataScience', 'id': 3}, {'name': 'is', 'id': 8}]

Example 2: Input:

test_list = ["CampusX", 10] key_list = ["name", "id"]

Output: [{'name': 'CampusX', 'id': 10}]

program

Convert a list of Tuples into Dictionary. Example 1:

Input:

[("akash", 10), ("gaurav", 12), ("anand", 14), ("suraj", 20), ("akhil", 25), ("ashish", 30)] Output: {'akash': [10], 'gaurav': [12], 'anand': [14], 'suraj': [20], 'akhil': [25], 'ashish': [30]}

Example 2:

Input:

[('A', 1), ('B', 2), ('C', 3)]

Output:

{'A': [1], 'B': [2], 'C': [3]}

In [97]:
```
L1 = [("akash", 10), ("gaurav", 12), ("anand", 14), ("suraj", 20), ("akhil",

L = [('A', 1), ('B', 2), ('C', 3)]
d = {}
for i,j in L:
    d[i] = [j]
print(d)
```

```
{'A': [1], 'B': [2], 'C': [3]}
```

program

Sort Dictionary key and values List.

Example 1:

Input:

{'c': [3], 'b': [12, 10], 'a': [19, 4]}

Output:

{'a': [4, 19], 'b': [10, 12], 'c': [3]}

Example 2:

Input:

{'c': [10, 34, 3]}

Output:

{'c': [3, 10, 34]}

In [98]:
```
d = {'c': [3], 'b': [12, 10], 'a': [19, 4]}
res = {}
for i in sorted(d):
    res[i] = sorted(d[i])
print(res)
```

```
{'a': [4, 19], 'b': [10, 12], 'c': [3]}
```

In [101]:
```
s="acd bcd avf ght dfg bgh kju avf ghv bvf ujh kju bgh avf nhj bgh bvf cdf a
d={}
for i in s.split():
    d[i[0]]=d.get(i[0],0)+1
print(d)
```

```
{'a': 5, 'b': 6, 'g': 3, 'd': 2, 'k': 3, 'u': 1, 'n': 1, 'c': 2, 's': 1,
'f': 1, 'x': 1, 'z': 1}
```

In [103]:
```
s="acd bcd avf ght dfg bgh kju avf ghv bvf ujh kju bgh avf nhj bgh bvf cdf a
d={}
for i in s.split():
    if i[0] not in d.keys():
        d[i[0]]=[]
        d[i[0]].append(i)
    else:
        d[i[0]].append(i)
print(d)
```

```
{'a': ['acd', 'avf', 'avf', 'avf', 'avg'], 'b': ['bcd', 'bgh', 'bvf', 'bg
h', 'bgh', 'bvf'], 'g': ['ght', 'ghv', 'gty'], 'd': ['dfg', 'der'], 'k':
['kju', 'kju', 'kju'], 'u': ['ujh'], 'n': ['nhj'], 'c': ['cdf', 'cdf'],
's': ['sde'], 'f': ['frt'], 'x': ['xsd'], 'z': ['zse']}
```

set

A set is an unordered collection of items.

Every set element is unique (no duplicates) and must be immutable (cannot be changed).

However, a set itself is mutable. We can add or remove items from it.

Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.

Characterstics: Unordered ,Mutable ,No Duplicates ,Can't contain mutable data types

creating set

In [107]:
```
# empty
s = set()
print(s)
print(type(s))
s={}
print(s)
print(type(s))
# 1D and 2D
s1 = {1,2,3}
print(s1)
#s2 = {1,2,3,{4,5}}
#print(s2)
# homo and hetro
s3 = {1,'hello',4.5,(1,2,3)}
print(s3)
# using type conversion

s4 = set([1,2,3])
print(s4)
# duplicates not allowed
s5 = {1,1,2,2,3,3}
print(s5)
# set can't have mutable items
s6 = {1,2,[3,4]}
print(s6)
```

```
set()
<class 'set'>
{}
<class 'dict'>
{1, 2, 3}
{1, (1, 2, 3), 4.5, 'hello'}
{1, 2, 3}
{1, 2, 3}

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-107-6e6f04bba55c> in <module>
     22 print(s5)
     23 # set can't have mutable items
---> 24 s6 = {1,2,[3,4]}
     25 print(s6)

TypeError: unhashable type: 'list'
```

In [108]:
```
{1,2,3}=={3,1,2}
```

Out[108]:
```
True
```

Accessing Items not allow using indexing or slicing

In [111]:
```
#Adding Items: add, update
S = {1,2,3,4}
# add
S.add(5)
print(S)
S.update([5,6,7])
print(S)
```

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5, 6, 7}
```

In [113]:
```
#Deleting Items:  del, discard,remove ,pop,  clear
# del
s = {1,2,3,4,5}
print(s)
del s
print(s)
```

```
{1, 2, 3, 4, 5}

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-113-57976383cc0b> in <module>
      4 print(s)
      5 del s
----> 6 print(s)

NameError: name 's' is not defined
```

In [114]:
```
#discard
s = {1,2,3,4,5}
print(s)
s.discard(3)
print(s)
s.discard(50)
print(s)
```

```
{1, 2, 3, 4, 5}
{1, 2, 4, 5}
{1, 2, 4, 5}
```

In [115]:
```
#remove
s = {1,2,3,4,5}
print(s)
s.remove(5)
print(s)
s.remove(50)
print(s)
```

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4}

---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-115-8ea2687aa962> in <module>
      4 s.remove(5)
      5 print(s)
----> 6 s.remove(50)
      7 print(s)

KeyError: 50
```

In [123]:
```
#pop
s = {0,1,7,2,3,4,5,8}
print(s)
s.pop()
print(s)
```

```
{0, 1, 2, 3, 4, 5, 7, 8}
{1, 2, 3, 4, 5, 7, 8}
```

In [124]:
```
#clear
s = {7,2,3,4,5}
print(s)
s.clear()
print(s)
```

```
{2, 3, 4, 5, 7}
set()
```

Set Operation

union(|) ,intersection(&) ,Difference(-),Symmetric Difference(^), Membership Test, Iteration

In [125]:
```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}

# Union(|)
print(s1 | s2,"Union(|)")
# Intersection(&)
print(s1 & s2,"Intersection(&)")
# Difference(-)
print(s1 - s2,"s1-s2 Difference(-)")
print(s2 - s1,"s2-s1 Difference(-)")
# Symmetric Difference(^)
print(s1 ^ s2,"Symmetric Difference(^)")
# Membership Test print(1 not in s1) # Iteration
for i in s1:
    print(i)
```

```
{1, 2, 3, 4, 5, 6, 7, 8} Union(|)
{4, 5} Intersection(&)
{1, 2, 3} s1-s2 Difference(-)
{8, 6, 7} s2-s1 Difference(-)
{1, 2, 3, 6, 7, 8} Symmetric Difference(^)
1
2
3
4
5
```

Set Functions

len ,sum, min, max, sorted

In [131]:
```
# len/sum/min/max/sorted
s = {3,1,4,5,2,7}
print(len(s))
print(min(s))
print(max(s))
print(sorted(s,reverse=True))
```

```
6
1
7
[7, 5, 4, 3, 2, 1]
```

In [2]:
```
s = {3,1,4,5,2,7}
s1=sum(s)
print(s1)
```

```
22
```

In [1]:
```
my_set = {1, 2, 3, 4, 5}
total = sum(my_set)
print(total)
```

```
15
```

In [135]:
```
s = {10,20,30}
s1 = s.copy()
print(s)
print(s1)
```

```
{10, 20, 30}
{10, 20, 30}
```

Set Comprehension

In [3]:
```
{i**2 for i in range(1,11) if i>5}
```

Out[3]:
```
{36, 49, 64, 81, 100}
```

Write a program to find set of common elements in three lists using sets.

Input : ar1 = [1, 5, 10, 20, 40, 80]

ar2 = [6, 7, 20, 80, 100]

ar3 = [3, 4, 15, 20, 30, 70, 80, 120]

Output : [80, 20]

In [5]:
```
ar1 = [1, 5, 10, 20, 40, 80]
ar2 = [6, 7, 20, 80, 100]
ar3 = [3, 4, 15, 20, 30, 70, 80, 120]
s1 = set(ar1)
s2 = set(ar2)
s3 = set(ar3)
result = list((s1 & s2) & s3)
print(result)
```

```
[80, 20]
```

In [6]:
```
s1 = {1,2,3,4,5}
s2 = {3,4,5}
s1.issuperset(s2)
```

Out[6]:
```
True
```

In [7]:
```
s1 = {1,2,3,4,5}
s2 = {3,4,5}
s2.issubset(s1)
```

Out[7]:
```
True
```

Frozenset

In [10]:
```
#Frozen set is just an immutable version of a Python set object
# create frozenset
fs1 = frozenset([1,2,3])
print(fs1)
fs2 = frozenset([3,4,5])
print(fs2)
print(fs1 | fs2)
```

```
frozenset({1, 2, 3})
frozenset({3, 4, 5})
frozenset({1, 2, 3, 4, 5})
```

In [14]:
```
fs = frozenset([1,2,frozenset([3,4])])
print(fs)
```

```
frozenset({frozenset({3, 4}), 1, 2})
```

In [2]:
```
l1 = ["eat", "sleep", "repeat"]
s1 = "gOOD"

# creating enumerate objects
obj1 = enumerate(l1)
obj2 = enumerate(s1)

print ("Return type:", type(obj1))
print (list(enumerate(l1)))

# changing start index to 2 from 0
print (list(enumerate(s1, 2)))
```

```
Return type: <class 'enumerate'>
[(0, 'eat'), (1, 'sleep'), (2, 'repeat')]
[(2, 'g'), (3, 'O'), (4, 'O'), (5, 'D')]
```

In [5]:
```
alph = ["a", "b", "c", "d"]
ralph = reversed(alph)
print(ralph)# return object
print(list(ralph))

t=(1,2,3)
print(tuple(reversed(t)))
```

```
<list_reverseiterator object at 0x0000024F22280AC0>
['d', 'c', 'b', 'a']
(3, 2, 1)
```

In [ ]: